



Read this paper fully and carefully. I don't understand Section 3.3 at all. But it seems Section 3.3 is very important for the methods the authors use and for the success of the drifting! So please explain to me how does Section 3.3 fits into the overall methods and what does Section 3.3 really mean step-by-step.

已思考 52s >

Section 3.3 is the **bridge between “what motion derivatives I want” and “what actuator commands I must send.”** Sections 3.1–3.2 say: “To drift along the path, I want a certain velocity-vector rotation rate $\dot{\phi}_{des}$, and I want a certain yaw acceleration $\dot{r}_{des}$.” Section 3.3 answers: “What steering angle δ and rear tire thrust direction/force make the car actually produce those two derivatives?”  ds_142_02_021004 (2)

1. The overall controller idea before Section 3.3

The paper separates the drifting-control problem into layers.

At the top level, the controller wants to track two things:

$$e \rightarrow 0$$

lateral path error goes to zero, and

$$\beta \rightarrow \beta_{ref}$$

sideslip follows the desired drifting angle.

The clever idea is:

$$\dot{\phi} = \dot{\beta} + r$$

where ϕ is the direction of the **velocity vector**, β is sideslip, and r is body yaw rate.

So the controller uses:

$$\dot{\phi}$$

to control the path, because the car's CG moves in the direction of the velocity vector, not necessarily in the direction the body is pointing.

Then it uses:

$$r$$

relative to $\dot{\phi}$ to control sideslip.

That is the key drift insight: during drifting, the body can point one way, while the velocity vector points another way.

So Section 3.2 produces two “desired derivative commands”:

$$\dot{\phi}_{des}$$

for path tracking, and

$$\dot{r}_{des}$$

for sideslip/yaw stabilization.

But those are not actuator commands. You cannot directly command $\dot{\phi}$ or $\dot{r}$. You can only command steering and rear drive torque/wheelspeed.

That is why Section 3.3 is needed.

2. What Section 3.3 does

Section 3.3 takes the desired pair

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

and converts it into physical inputs:

$$\delta$$

steering angle, and

$$\gamma$$

rear tire thrust angle, or equivalently a rear longitudinal/lateral force combination.

The paper uses the nonlinear bicycle model from Eq. 5. That model says: given current vehicle state (V, β, r) , and given steering/rear tire forces, the car produces some:

$$\dot{r}, \quad \dot{\beta}, \quad \dot{V}$$

Since

$$\dot{\phi} = \dot{\beta} + r$$

the model can also predict:

$$\dot{\phi}$$

So Section 3.3 numerically solves the inverse problem:

Find δ and rear force/thrust angle such that

$$\dot{\phi}(\delta, F_{xr}) = \dot{\phi}_{des}$$

$$\dot{r}(\delta, F_{xr}) = \dot{r}_{des}$$

This is the “nonlinear model inversion.”

In plain language:

The controller first decides how fast the velocity vector should rotate and how fast the body yaw rate should change. Then Section 3.3 finds the steering and rear sliding-tire force direction that make those two things happen.

3. Why speed V is not directly controlled

The bicycle model has three state derivatives:

$$\dot{\phi}, \quad \dot{r}, \quad \dot{V}$$

But the controller has only two main inputs:

$$\delta, \quad \gamma$$

or approximately

$$\delta, \quad F_{xr}$$

So it can directly choose only two derivatives. The authors choose to directly control:

$$\dot{\phi}_{des}$$

for path tracking, and

$$\dot{r}_{des}$$

for sideslip stabilization.

That leaves:

$$\dot{V}$$

uncontrolled.

This is very important. In normal driving, we usually think:

steering controls path, throttle controls speed.

But in drifting, throttle does not simply control speed. Because the rear tires are saturated, changing rear drive force changes the **direction** of the rear tire force, which strongly affects yaw and sideslip.

So the paper effectively says:

During drifting, steering and throttle must be used together to control path and sideslip. Speed becomes an internal consequence of the coupled dynamics.

That sounds dangerous unless $\dot{V}$ naturally becomes stable. Section 3.3 is where the authors analyze why that can work.

4. What “velocity zero-dynamics” means here

“Zero dynamics” means: suppose the controlled errors have already gone to zero.

So assume:

$$e = 0$$

$$\Delta\phi = 0$$

$$\beta = \beta_{ref}$$

$$r = r_{syn}$$

Then the path and sideslip objectives are perfectly satisfied. What happens to the remaining uncontrolled variable V ?

That remaining speed behavior is the **velocity zero-dynamics**.

At zero tracking error, the desired velocity-vector rotation rate becomes:

$$\dot{\phi}_{des} = \dot{\phi}_{ref}$$

and for a path with curvature κ_{ref} ,

$$\dot{\phi}_{ref} = V\kappa_{ref}$$

This is crucial.

It means: for the same path curvature, if the vehicle speed V is higher, the velocity vector must rotate faster to stay on the path.

So the commanded $\dot{\phi}_{des}$ grows linearly with speed.

Section 3.3 asks:

When the controller asks for a larger $\dot{\phi}$, does the resulting physics make the car slow down or speed up?

For speed stability, we want:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

along the relevant constant- $\dot{r}$ contour.

That means:

- if V is too high, then $\dot{\phi}_{des} = V\kappa$ is high;
- the model inversion asks the car to generate that high $\dot{\phi}$;
- on the selected branch, this causes $\dot{V} < 0$;
- so the car slows down.

And if V is too low:

- $\dot{\phi}_{des}$ is lower;
- the selected input solution causes $\dot{V} > 0$;
- so the car speeds up.

That is why speed can be stable even though speed is not explicitly regulated.

5. What the “feasible surface” means

For a fixed vehicle state (V, β, r) , imagine sweeping all possible steering angles and rear force/thrust commands.

For each input pair, the vehicle model predicts:

$$\dot{r}, \dot{\phi}, \dot{V}$$

Because there are two inputs, all possible derivative triples form a 2D surface in 3D space:

$$(\dot{r}, \dot{\phi}, \dot{V})$$

That is the “surface of possible state derivatives” shown in Fig. 4 on page 4.

The controller wants a specific pair:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

but it does not directly choose $\dot{V}$. So geometrically, it is looking for the point on that feasible surface that has the desired $\dot{\phi}$ and $\dot{r}$. The height of that point gives the resulting $\dot{V}$.

This is a very nice way to understand Section 3.3:

The controller chooses a point on the feasible derivative surface by specifying $\dot{\phi}$ and $\dot{r}$. The leftover vertical coordinate is $\dot{V}$.

6. Why there can be multiple inverse solutions

The feasible surface is curved and folded. Therefore, the same desired pair

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

may correspond to more than one possible input pair.

That means nonlinear inversion may not be one-to-one.

Physically, there may be two different combinations of steering and rear tire force that produce the same yaw acceleration and course-rate, but with different acceleration/deceleration behavior.

One branch may make speed stable.

Another branch may make speed unstable.

So the authors need to choose the correct branch of the inverse mapping.

This is why they introduce the “separatrix.”

7. What the separatrix means

The separatrix is the boundary where the mapping from inputs to desired derivatives becomes singular or folds over.

Mathematically, Eq. 15 is a Jacobian determinant condition:

$$\frac{\partial \dot{\phi}}{\partial \delta} \frac{\partial \dot{r}}{\partial F_{xr}} - \frac{\partial \dot{\phi}}{\partial F_{xr}} \frac{\partial \dot{r}}{\partial \delta} = 0$$

This is the condition where the mapping

$$(\delta, F_{xr}) \mapsto (\dot{\phi}, \dot{r})$$

loses local invertibility.

In simpler words:

At the separatrix, small changes in steering and rear force no longer produce independent changes in $\dot{\phi}$ and $\dot{r}$. The feasible surface folds there.

The authors then choose the “upper” side of the feasible surface, where the speed zero-dynamics are stable.

That is why Section 3.3 says constraining model inversion to this upper surface makes the mapping one-to-one and gives stable velocity behavior.

8. Why this is essential for successful drifting

Without Section 3.3, the controller would only say:

$$\dot{\phi}_{des}, \quad \dot{r}_{des}$$

but it would not know what steering and rear tire commands to use.

Even worse, if the inverse model chose the wrong branch, the car might still initially track path and sideslip, but the uncontrolled speed dynamics could diverge.

For example:

- if the car is too fast and the inversion branch makes $\dot{V} > 0$, the car accelerates more;
- then $\dot{\phi}_{des} = V\kappa$ grows;
- the controller asks for even more aggressive motion;
- the drift can become infeasible or unstable.

The correct branch does the opposite:

- too fast $\rightarrow$ controller-induced $\dot{V} < 0$;
- too slow $\rightarrow$ controller-induced $\dot{V} > 0$.

So Section 3.3 is not just an implementation detail. It is one of the main reasons the method can drift along a general path without explicitly controlling speed.

9. What Section 3.3.1 means: projection into feasible space

Sometimes the desired pair

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

is impossible. The car cannot generate arbitrary yaw acceleration and velocity-vector rotation rate because steering, friction, and tire saturation limit the feasible space.

So Section 3.3.1 says they handle infeasible commands in a simple way:

First, saturate:

$$\dot{\phi}_{des}$$

so it lies within feasible values.

Then compute the corresponding sideslip/yaw command.

Then project the final pair

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

into the feasible space along the line:

$$\dot{r} = \dot{r}_{des}$$

This means they try harder to preserve the desired yaw acceleration than the exact desired path-tracking course rate.

In plain language:

If the car cannot do everything, prioritize stabilizing yaw/sideslip over perfectly reducing lateral path error.

That makes sense because sideslip/yaw is the more unstable part during drifting. If you lose yaw/sideslip stability, the drift collapses. If you temporarily accept slightly worse lateral error, you may still recover.

10. The complete control flow with Section 3.3 included

At each control step:

1. Measure vehicle state:

$$V, \beta, r, e, \Delta\phi$$

2. Use path-tracking dynamics to compute:

$$\dot{\phi}_{des}$$

3. Use sideslip stabilization to compute:

$$r_{syn}$$

4. Use yaw-rate inner-loop dynamics to compute:

$$\dot{r}_{des}$$

5. Section 3.3 solves the nonlinear inverse problem:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \rightarrow (\delta, \gamma)$$

6. Section 4 maps rear thrust angle γ to desired rear wheelspeed.

7. The low-level wheelspeed controller commands motor torque.

So Section 3.3 is the middle layer:

desired drift motion $\rightarrow$ desired state derivatives $\rightarrow$ Section 3.3 nonlinear model inversion $\rightarrow$ steering + rear thru

My interpretation in one sentence

Section 3.3 says: **during drifting, do not separately use steering for path and throttle for speed; instead, use steering and rear tire thrust together to command the velocity-vector rotation rate and yaw acceleration, then deliberately choose the inverse-solution branch where the leftover speed dynamics naturally stabilize themselves.**

Double check your answers carefully, and then explain to me in even more details, especially from Point 4 and onwards!

已思考 42s >

Yes — after double-checking, my previous explanation was basically correct, but I should refine one important detail:

In Section 3.3, the authors say the inverse model maps

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

to

$$\delta \quad \text{and} \quad \gamma$$

where δ is steering angle and γ is rear thrust angle. Internally, because the rear tire is assumed saturated, this can also be parameterized through F_{xr} , because once F_{xr} is chosen, F_{yr} follows from the friction-circle relation. Then γ is later mapped to desired wheelspeed in Section 4. So my previous “ δ and F_{xr} ” explanation was physically right, but the paper’s controller-output language is more accurately “ δ and rear thrust angle γ .” ds_142_02_021004 (2)

Now let me go much deeper from the velocity-zero-dynamics part onward.

1. The central difficulty Section 3.3 solves

Before Section 3.3, the controller has already decided two things:

$$\dot{\phi}_{des}$$

and

$$\dot{r}_{des}$$

Here:

- ϕ is the **course angle**, meaning the direction of the vehicle velocity vector.
- $\dot{\phi}$ is the **course rate**, meaning how fast the velocity vector rotates.
- r is the body yaw rate.
- $\dot{r}$ is the yaw acceleration.

The paper’s key kinematic relation is:

$$\dot{\phi} = \dot{\beta} + r$$

or equivalently:

$$\dot{\beta} = \dot{\phi} - r$$

This is very important.

During normal driving, β is small, so the car body roughly points where it is going. But during drifting, β is large, so the body direction and velocity direction are very different.

That means the controller cannot think only in terms of yaw angle. It must think separately about:

where the car is moving

and

where the car body is pointing

So the controller uses:

$$\dot{\phi}_{des}$$

to make the **velocity vector follow the path**, and uses:

$$\dot{r}_{des}$$

to make the **body yaw dynamics stabilize sideslip**.

Section 3.3 is where these desired derivatives are converted into real physical inputs through nonlinear model inversion. The authors explicitly state that the force-based nonlinear single-track model is numerically inverted to map the desired state-derivative pair to steering and rear thrust angle.  ds_142_02_021004 (2)

2. Why speed becomes uncontrolled

The model has three important motion derivatives:

$$\dot{\phi}, \quad \dot{r}, \quad \dot{V}$$

But the controller has only two main inputs:

$$\delta, \quad \gamma$$

or equivalently, steering and rear sliding-tire force direction.

So the controller cannot independently command all three:

$$\dot{\phi}, \dot{r}, \dot{V}$$

It chooses to command:

$$\dot{\phi}$$

for path tracking, and

$$\dot{r}$$

for yaw/sideslip stabilization.

That leaves:

$$\dot{V}$$

as a consequence.

This is the exact meaning of the paper's sentence saying that because there are two desired state derivatives and two inputs, the total acceleration $\dot{V}$ is an uncontrolled variable set by the choice of $(\dot{\phi}_{des}, \dot{r}_{des})$.

 ds_142_02_021004 (2)

This is a very deep point. In ordinary driving, you usually think:

$$\delta \rightarrow \text{path}$$

$$\text{throttle/brake} \rightarrow \text{speed}$$

But in drifting, this separation breaks down. Rear throttle does not only accelerate the car. Because the rear tires are saturated, changing rear wheel speed changes the rear tire slip direction, which changes the rear

force direction, which strongly affects yaw and sideslip.

So during drifting, throttle is not mainly a speed-control actuator. It becomes a yaw/sideslip/path-control actuator.

That is why the authors say there is a “shuffling of objectives”: in drifting, steering and throttle work together to control path and sideslip, while velocity is left to the coupled dynamics.  ds_142_02_021004 (2)

3. Why leaving speed uncontrolled is not automatically wrong

At first, this seems dangerous.

You might ask:

If speed is not controlled, why does the vehicle not accelerate forever or slow down to zero?

The answer is the main content of Section 3.3:

Under the chosen inverse-model branch, the uncontrolled speed dynamics can become naturally stable.

This is what the authors call analyzing V as a **zero-dynamic**.

4. What “zero dynamics” means here, carefully

In nonlinear control, “zero dynamics” means:

Suppose the controlled outputs have been driven exactly to their desired values. What happens to the internal state that was not directly controlled?

Here, the directly controlled objectives are path error and sideslip/yaw behavior.

The paper says the controller explicitly drives:

$$e \rightarrow 0$$

$$\Delta\phi \rightarrow 0$$

$$e_r = r - r_{syn} \rightarrow 0$$

$$e_\beta = \beta - \beta_{ref} \rightarrow 0$$

So imagine the controller is doing a perfect job. Then:

- lateral error is zero,
- course error is zero,
- sideslip error is zero,
- yaw-rate synthetic tracking error is zero.

Now the question becomes:

What happens to V ?

That remaining V -behavior is the **velocity zero-dynamics**.

It is called “zero” dynamics because the controlled errors are set to zero.

5. At zero error, why $\dot{\phi}_{des} = V\kappa_{ref}$

This is the most important equation in Section 3.3.

From path geometry, if a vehicle moves along a path with curvature κ_{ref} , then the course angle must rotate at:

$$\dot{\phi}_{ref} = V\kappa_{ref}$$

This is just the standard relation:

$$\text{yaw/course rate} = \text{speed} \times \text{curvature}$$

For example:

- If the car moves slowly around a circle, the velocity direction rotates slowly.
- If the car moves quickly around the same circle, the velocity direction must rotate faster.
- If the path curvature is larger, the velocity direction must also rotate faster.

At zero path error and zero course error, the desired course-rate becomes:

$$\dot{\phi}_{des} = \dot{\phi}_{ref} = V\kappa_{ref}$$

This is Eq. 14 in the paper. The authors simplify the more general curvilinear expression to $V\kappa_{ref}$ when $e = 0$ and $\Delta\phi = 0$. [ds_142_02_021004 \(2\)](#)

This equation couples speed directly into the path-tracking command.

So even though the controller is not directly commanding speed, speed still affects the controller because speed changes the required $\dot{\phi}_{des}$.

6. The stabilizing-speed idea

Now suppose the car is supposed to drift around a given curvature.

At the correct speed, the controller asks for the correct course-rate:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

Now imagine the car becomes too fast.

Then V increases, so:

$$\dot{\phi}_{des}$$

also increases.

The controller now asks the vehicle to rotate its velocity vector faster.

Depending on the nonlinear vehicle dynamics, producing this larger $\dot{\phi}$ may require a combination of steering and rear thrust that causes:

$$\dot{V} < 0$$

So the vehicle decelerates back toward the correct speed.

Likewise, if the car becomes too slow, then:

$$\dot{\phi}_{des}$$

decreases. The inverse model may select inputs that cause:

$$\dot{V} > 0$$

so the car accelerates back.

This is the desired speed-stability mechanism:

$$V \uparrow \Rightarrow \dot{\phi}_{des} \uparrow \Rightarrow \dot{V} < 0 \Rightarrow V \downarrow$$

and

$$V \downarrow \Rightarrow \dot{\phi}_{des} \downarrow \Rightarrow \dot{V} > 0 \Rightarrow V \uparrow$$

That is why the paper says velocity should be stable if:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

along the appropriate constant- r contour. [ds_142_02_021004 \(2\)](#)

This condition means:

If the controller asks for a larger course-rate, the resulting vehicle acceleration should become smaller or

more negative.

That is exactly a negative-feedback effect on speed.

7. Why the condition is “along constant $\dot{r}$ ” rather than globally

The controller is not only commanding $\dot{\phi}$. It is commanding both:

$$\dot{\phi}_{des}$$

and

$$\dot{r}_{des}$$

So when analyzing how $\dot{V}$ changes with $\dot{\phi}$, we should hold $\dot{r}$ fixed.

That is why the paper says:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

along contours of constant $\dot{r}$.

This means:

For the same desired yaw acceleration, if the desired course-rate is increased, does the resulting model inversion make the vehicle decelerate?

If yes, speed is locally stable.

If no, speed may be unstable.

8. What the feasible surface really is

For a fixed vehicle state:

$$(V, \beta, r)$$

imagine trying many possible actuator inputs:

$$\delta, \gamma$$

or equivalently:

$$\delta, F_{xr}$$

For each input pair, the vehicle model predicts one derivative triple:

$$(\dot{r}, \dot{\phi}, \dot{V})$$

Because there are two inputs, all possible derivative triples form a 2D surface in 3D derivative space:

$$(\dot{r}, \dot{\phi}, \dot{V})$$

That is the “surface of possible state derivatives” shown in Fig. 4 on page 4.

This surface is the set of all motion derivatives the car can physically generate at that instant.

The controller wants a certain:

$$\dot{\phi}_{des}$$

and a certain:

$$\dot{r}_{des}$$

So geometrically, it is asking:

Find the point on the feasible surface whose horizontal coordinates are $(\dot{\phi}_{des}, \dot{r}_{des})$. The vertical coordinate at that point is the resulting $\dot{V}$.

That is why $\dot{V}$ is “uncontrolled” but not arbitrary. It is the remaining coordinate of the feasible-surface point selected by the inverse model.

9. Why the feasible surface can fold

The feasible surface is nonlinear because tire forces are nonlinear and saturated.

So the mapping:

$$(\delta, F_{xr}) \rightarrow (\dot{\phi}, \dot{r}, \dot{V})$$

is curved.

If you project this surface onto the $(\dot{\phi}, \dot{r})$ plane, the same desired pair can correspond to multiple points on the full surface.

That means the inverse problem may have multiple solutions:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \rightarrow (\delta_1, F_{xr,1})$$

and

$$(\dot{\phi}_{des}, \dot{r}_{des}) \rightarrow (\delta_2, F_{xr,2})$$

Both may produce the same $\dot{\phi}$ and $\dot{r}$, but different $\dot{V}$.

This is extremely important.

One solution branch may give:

$$\dot{V} < 0$$

while another gives:

$$\dot{V} > 0$$

for the same desired $\dot{\phi}$ and $\dot{r}$.

So just “solving the nonlinear inverse model” is not enough. The controller must choose the correct branch.

10. What the “upper surface” means

The paper says that if the separatrix lies below the $\dot{V} = 0$ plane, constraining inversion to the “upper” surface should give stable velocity zero-dynamics.  ds_142_02_021004 (2)

This wording is slightly confusing because “upper” is not necessarily visually obvious unless you look at Figs. 4–6.

The authors are looking at the feasible derivative surface and identifying a boundary called the **separatrix**. Above this boundary, the slope condition is favorable:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

So “upper surface” means the branch of the feasible surface on the side of the separatrix where the speed feedback has the correct sign.

It does not simply mean “larger $\dot{V}$ ” everywhere. It means the selected branch of the folded surface that gives:

1. locally one-to-one inverse mapping, and
 2. stable speed zero-dynamics.
-

11. What the separatrix means physically

Mathematically, the separatrix is where the mapping from actuator inputs to controlled derivatives becomes singular.

The mapping is:

$$(\delta, F_{xr}) \mapsto (\dot{\phi}, \dot{r})$$

Its Jacobian is:

$$J = \begin{bmatrix} \frac{\partial \dot{\phi}}{\partial \delta} & \frac{\partial \dot{\phi}}{\partial F_{xr}} \\ \frac{\partial \dot{r}}{\partial \delta} & \frac{\partial \dot{r}}{\partial F_{xr}} \end{bmatrix}$$

The determinant is:

$$\det J = \frac{\partial \dot{\phi}}{\partial \delta} \frac{\partial \dot{r}}{\partial F_{xr}} - \frac{\partial \dot{\phi}}{\partial F_{xr}} \frac{\partial \dot{r}}{\partial \delta}$$

The separatrix occurs when:

$$\det J = 0$$

This is exactly Eq. 15 in the paper.

 ds_142_02_021004 (2)

Physically, $\det J = 0$ means the two actuators are no longer independently controlling the two desired derivatives.

For example, away from the separatrix:

- changing steering may affect $\dot{\phi}$ in one direction,
- changing rear thrust may affect $\dot{r}$ in another independent direction,

so you can solve for both desired derivatives.

At the separatrix, the two actuator effects become linearly dependent in the $(\dot{\phi}, \dot{r})$ plane. The surface is folding over. The inverse mapping becomes ambiguous or ill-conditioned.

So the separatrix is the boundary between different inverse-solution branches.

12. Why Eq. 15 is not about vehicle stability directly

This is a subtle point.

Eq. 15 does **not** directly say:

vehicle stable

or

vehicle unstable

It says:

the input-to-output derivative map is singular

That singular boundary is then used to separate branches of the feasible surface.

The stability argument comes from choosing the branch where:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

and where the surface crosses $\dot{V} = 0$ appropriately.

So the logical structure is:

1. The inverse mapping may have multiple branches.
2. The separatrix identifies the boundary between branches.
3. One branch has favorable $\dot{V}$ -versus- $\dot{\phi}$ slope.
4. Choosing that branch gives stable velocity zero-dynamics.

That is the real meaning of the separatrix discussion.

13. What “zero-crossing of $\dot{V}$ ” means

The paper says velocity should be stable as long as:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

and $\dot{V}$ has a zero-crossing along:

$$\dot{r} = \dot{r}_{ref}$$

or more generally along contours of constant $\dot{r}$.

ds_142_02_021004 (2)

This has two requirements.

First, you need an equilibrium speed:

$$\dot{V} = 0$$

If $\dot{V}$ never crosses zero, then there is no speed where the car naturally settles.

Second, the slope must have the correct sign around that crossing.

A simple scalar analogy is:

$$\dot{V} = -k(V - V^*)$$

where $k > 0$. Then:

- if $V > V^*$, $\dot{V} < 0$;
- if $V < V^*$, $\dot{V} > 0$.

That is stable.

In the paper, the analysis is not directly $\dot{V}$ versus V . It is $\dot{V}$ versus $\dot{\phi}$. But at zero error:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

So changing V changes $\dot{\phi}_{des}$. Therefore the slope:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

becomes a speed-stabilizing mechanism.

14. A concrete example

Suppose the car is drifting around a left-hand path with fixed curvature:

$$\kappa_{ref} > 0$$

At the desired drift equilibrium, maybe:

$$V = 10 \text{ m/s}$$

and:

$$\kappa_{ref} = 0.1 \text{ m}^{-1}$$

Then:

$$\dot{\phi}_{des} = V\kappa_{ref} = 1.0 \text{ rad/s}$$

Now imagine the car accidentally speeds up to:

$$V = 12 \text{ m/s}$$

Then:

$$\dot{\phi}_{des} = 1.2 \text{ rad/s}$$

The controller asks for a larger velocity-vector rotation rate.

On the chosen feasible-surface branch, asking for this larger $\dot{\phi}$ causes:

$$\dot{V} < 0$$

because the required steering/rear-thrust combination consumes more longitudinal motion, rotates the velocity vector harder, and slows the vehicle.

So speed comes back down.

If instead the car slows to:

$$V = 8 \text{ m/s}$$

then:

$$\dot{\phi}_{des} = 0.8 \text{ rad/s}$$

The controller asks for less aggressive course rotation. On the chosen branch, that produces:

$$\dot{V} > 0$$

so the car accelerates back.

This is the core physical intuition.

15. Why this is especially possible during drifting

During normal driving, longitudinal and lateral tire forces are often treated as somewhat separable:

- lateral force turns the car,
- longitudinal force changes speed.

But during drifting, the rear tire is saturated. The rear force magnitude is approximately fixed by:

$$\mu F_z$$

and changing wheel speed changes the direction of the rear tire force.

So longitudinal and lateral effects are strongly coupled.

When the rear thrust angle changes, you are not just changing acceleration. You are also changing yaw moment and sideslip behavior.

This coupling is what makes Section 3.3 possible.

The controller does not fight the coupling. It uses it.

That is why the authors later say the method is about “leveraging—rather than fighting—the coupled velocity dynamics.”  ds_142_02_021004 (2)

16. Why Fig. 5 matters

Fig. 5 plots:

$$\dot{\phi}$$

against:

$$\dot{V}$$

along contours of constant:

$$\dot{r}$$

The red separatrix divides the feasible curve into two regions.

The important question is:

On the selected side, when $\dot{\phi}$ increases, does $\dot{V}$ decrease?

If yes:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

Then speed is stable through the zero-dynamics mechanism.

So Fig. 5 is not just a pretty plot. It is the geometric evidence for the speed-stability argument.

17. Why Fig. 6 matters

Fig. 6 plots:

$$\dot{r}$$

versus:

$$\dot{\phi}$$

with contours of:

$$\dot{V}$$

This figure is about inverse-model uniqueness.

The controller wants to select a point in the $(\dot{\phi}, \dot{r})$ plane. If the mapping folds, one desired pair might correspond to multiple actuator combinations.

By staying on the correct side of the separatrix, the authors make the mapping:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \rightarrow (\delta, \gamma)$$

approximately one-to-one.

This is important for implementation. If the inversion were not one-to-one, the numerical solver might jump between branches, creating discontinuous steering/throttle commands.

18. Why Fig. 7 matters

The authors admit that the feasible surface changes with state.

The shape of the feasible surface depends on:

$$V, \beta, r$$

So checking one example is not enough.

They numerically check a broad state volume:

$$0.2 < r < 1.25 \text{ rad/s}$$

$$3 < V < 30 \text{ m/s}$$

$$-5^\circ < \beta < 50^\circ$$

They find that the favorable condition generally holds except at very high yaw rates and very low speeds.

 ds_142_02_021004 (2)

That matters because the controller is intended for a general path, not just one fixed drift equilibrium.

So Section 3.3 is partly a robustness argument:

This inverse-solution branch should keep speed stable over a wide drifting state range, not merely at one circular drift equilibrium.

19. Section 3.3.1: what happens if the desired derivative pair is impossible

The desired pair:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

may be outside the feasible actuation space.

That means no combination of steering and rear thrust can produce both desired values at the current state.

This can happen because:

- steering angle is limited,
- tire friction is limited,
- rear tire force magnitude is limited by μF_z ,
- current speed/yaw/sideslip state may make the desired derivative physically impossible.

So Section 3.3.1 says they use a simple projection method.

The method is:

1. Saturate $\dot{\phi}_{des}$ so it lies within feasible $\dot{\phi}$ values.
2. Use that saturated $\dot{\phi}_{des}$ to compute the yaw/sideslip command.
3. Project the final pair $(\dot{\phi}_{des}, \dot{r}_{des})$ into the feasible space along the line:

$$\dot{r} = \dot{r}_{des}$$

The paper says this roughly prioritizes the unstable yaw/sideslip dynamics over the lateral-error dynamics.

ds_142_02_021004 (2)

This is logical. If the car cannot do everything, keeping sideslip/yaw stable is more urgent than reducing lateral path error immediately.

Path error can be corrected later. A spin may not be recoverable.

20. Why Section 3.3 is essential to the success of the drifting

Section 3.3 is essential for four reasons.

First, it converts abstract desired derivatives into actuator commands:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \rightarrow (\delta, \gamma)$$

Without this, Sections 3.1–3.2 are only a high-level desired-motion law.

Second, it handles the underactuation problem. There are three natural derivatives:

$$\dot{\phi}, \dot{r}, \dot{V}$$

but only two controlled inputs. Section 3.3 chooses to control path and sideslip, while letting speed become an internal zero-dynamic.

Third, it explains why speed does not need a separate controller. Speed is stabilized by the coupling between lateral and longitudinal dynamics at high sideslip.

Fourth, it chooses the correct branch of the nonlinear inverse map. Without that, the same desired $(\dot{\phi}, \dot{r})$ could be achieved by an input combination that makes speed unstable.

So the real contribution of Section 3.3 is not merely “we numerically invert a bicycle model.” The deeper contribution is:

The authors design the inversion so that the uncontrolled speed dynamics become stable rather than dangerous.

21. How I would rewrite Section 3.3 in plain language

Here is the whole section in plain language:

We want the car’s velocity vector to rotate at $\dot{\phi}_{des}$ so the car follows the path, and we want the body yaw acceleration $\dot{r}_{des}$ so sideslip stays stable. We use the nonlinear vehicle model to find the steering and rear tire thrust direction that produce those two derivatives. Since we only have two inputs, we cannot also command speed acceleration $\dot{V}$. But at zero tracking error, the desired course rate is $V \kappa_{ref}$, so if

speed changes, the controller automatically asks for a different course rate. On the correct branch of the feasible derivative surface, asking for a higher course rate produces deceleration, and asking for a lower course rate produces acceleration. Therefore speed is naturally stabilized. To make this work, we restrict the inversion to the branch of the feasible surface separated by the separatrix, where the inverse is one-to-one and the speed zero-dynamics have the right sign.

That is Section 3.3.

22. Connection to your own vehicle-control intuition

For your 3D ribbon-road OCP thinking, Section 3.3 is conceptually similar to choosing which outputs you control directly and which state becomes an internal dynamic.

Here the authors effectively choose:

$$\begin{aligned}y_1 &= e \\ y_2 &= \beta - \beta_{ref}\end{aligned}$$

and leave:

$$V$$

as an internal dynamic.

But they do not ignore V . They check whether the internal V -dynamics are stable under the chosen inversion branch.

That is the key lesson:

If a model is underactuated, you may not need to control every state directly, but you must understand the zero-dynamics of the states you leave uncontrolled.

In this paper, that uncontrolled state is speed. In your future 3D-road vehicle-control problem, similar questions could arise for vertical motion, roll/pitch, wheel loads, wheel speeds, or sideslip depending on your chosen outputs and inputs.

Thank you for your explanation, but I still feel not fully understand. So can you explain in the very best way and the most detailed way?

已思考 1m 27s >

The reason Section 3.3 feels hard is that it compresses **three different ideas** into a few paragraphs:

1. **Nonlinear model inversion:** convert desired motion derivatives into steering and rear tire thrust commands.
2. **Velocity zero-dynamics:** explain why speed can be left uncontrolled and still remain stable.
3. **Branch selection using the feasible surface/separatrix:** choose the correct inverse solution so the uncontrolled speed dynamics have the right sign.

The best mental picture is this:

In drifting, steering and throttle are no longer “path actuator” and “speed actuator.” Instead, steering and rear tire thrust work together to control **where the velocity vector goes** and **how the body yaws relative to it**. Speed is not directly controlled; it becomes the leftover internal dynamic. Section 3.3 explains why that leftover speed dynamic can stabilize itself, but only if the nonlinear inversion chooses the correct branch.

The paper itself says the controller uses the rotation rate of the velocity vector to track the path, yaw acceleration to stabilize sideslip, and then nonlinear model inversion plus wheelspeed control to realize those required state derivatives.  ds_142_02_021004 (2)

1. Start from the physical picture: drifting separates “where the car points” from “where the car moves”

During normal driving, the car body roughly points in the same direction as the velocity vector. So people casually think:

$$\text{car heading} \approx \text{velocity direction}$$

But during drifting, this is false.

The body may be pointing significantly inward/outward, while the center of mass moves along a different direction.

So the paper defines:

$$\phi = \text{course angle}$$

meaning the direction of the **velocity vector**, not the direction of the vehicle body.

The body yaw rate is:

$$r$$

The sideslip angle is:

$$\beta$$

The crucial kinematic relationship is:

$$\dot{\phi} = \dot{\beta} + r$$

or:

$$\dot{\beta} = \dot{\phi} - r$$

This one equation is almost the whole conceptual foundation.

It says:

The velocity vector rotates at rate $\dot{\phi}$. The body rotates at rate r . The difference between them changes sideslip.

So if you control $\dot{\phi}$, you control where the car is moving.

If you control r relative to $\dot{\phi}$, you control sideslip.

That is why the paper does **not** simply use yaw angle/yaw rate for path tracking. In drifting, yaw direction and motion direction are different.

2. Path tracking: why the controller wants $\dot{\phi}_{des}$

The vehicle's center of mass moves along its velocity vector. So if you want the CG to follow a path, you should control the direction of the velocity vector.

The paper defines lateral error e , course error $\Delta\phi$, and path curvature κ_{ref} . For small course error:

$$\dot{e} \approx V\Delta\dot{\phi}$$

and:

$$\ddot{e} \approx V\Delta\ddot{\phi}$$

So the lateral path error is controlled by controlling the **rate of change of course error**, which means controlling $\dot{\phi}$.

That is why Section 3.2 generates:

$$\dot{\phi}_{des}$$

The controller is effectively saying:

“To reduce lateral path error, I need the velocity vector to rotate at this rate.”

The paper explicitly defines $\dot{\phi}$ as the rotation rate of the velocity vector and uses it for path tracking.

ds_142_02_021004 (2)

3. Sideslip control: why the controller also wants $\dot{r}_{des}$

Now suppose the path-tracking part has chosen:

$$\dot{\phi}_{des}$$

But we also want:

$$\beta \rightarrow \beta_{ref}$$

From:

$$\dot{\beta} = \dot{\phi} - r$$

once $\dot{\phi}$ is chosen, the only way to control $\dot{\beta}$ is to control r .

If β is too small or too large, the controller chooses a desired yaw rate r_{syn} , called the synthetic yaw-rate input.

Then it drives the actual yaw rate r toward r_{syn} by commanding a desired yaw acceleration:

$$\dot{r}_{des}$$

So before Section 3.3, the controller has produced two desired derivative quantities:

$$\dot{\phi}_{des}$$

for path tracking, and

$$\dot{r}_{des}$$

for yaw/sideslip stabilization.

But these are not actuator commands.

You cannot send $\dot{\phi}_{des}$ to the steering rack.

You cannot directly command $\dot{r}_{des}$ with the motor.

So Section 3.3 is needed.

4. What Section 3.3 actually does

Section 3.3 solves this inverse problem:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \implies (\delta, \gamma)$$

where:

$$\delta = \text{front steering angle}$$

and

$$\gamma = \text{rear thrust angle}$$

The rear thrust angle γ describes the direction of the saturated rear tire force. In the paper's implementation, this is later mapped to desired wheel speed in Section 4.

So Section 3.3 is the middle block:

desired path/sideslip behavior

$\Downarrow$

$$\dot{\phi}_{des}, \dot{r}_{des}$$

$$\delta, \gamma$$



steering command and rear wheelspeed/torque command

The paper says the force-based nonlinear single-track model is numerically inverted to map the desired pair $(\dot{\phi}_{des}, \dot{r}_{des})$ to steering angle and rear thrust angle.  ds_142_02_021004 (2)

5. Why there are only two inputs but three important derivatives

The vehicle model predicts three important derivatives:

$$\dot{\phi}$$

course-rate, or velocity-vector rotation rate;

$$\dot{r}$$

yaw acceleration;

$$\dot{V}$$

speed acceleration.

But the controller has only two main inputs:

$$\delta$$

and

$$\gamma$$

So it can directly impose only two things.

The authors choose to directly impose:

$$\dot{\phi}_{des}$$

and:

$$\dot{r}_{des}$$

That leaves:

$$\dot{V}$$

uncontrolled.

This is not an accident. It is the central design choice.

They are saying:

In drifting, controlling path and sideslip is more important than directly controlling speed. We will use steering and rear thrust together to control path and sideslip, and then check whether speed naturally stabilizes as an internal dynamic.

That is the meaning of this sentence in Section 3.3: because there are two desired derivatives and two inputs, total acceleration $\dot{V}$ is an uncontrolled variable determined by the chosen $(\dot{\phi}_{des}, \dot{r}_{des})$.

 ds_142_02_021004 (2)

6. The most important physical interpretation: tire force has a “budget”

This is probably the easiest way to understand why speed can stabilize.

A vehicle's tire forces can be decomposed relative to the velocity vector:

$$F_{\parallel} = \text{force along velocity}$$

$F_{\perp}$ = force perpendicular to velocity

The parallel force changes speed:

$$m\dot{V} = F_{\parallel}$$

The perpendicular force rotates the velocity vector:

$$mV\dot{\phi} = F_{\perp}$$

So:

$$\dot{V} = \frac{F_{\parallel}}{m}$$
$$\dot{\phi} = \frac{F_{\perp}}{mV}$$

This is a very powerful interpretation.

To make $\dot{\phi}$ large, the car needs a large force component perpendicular to the velocity vector.

But tire force is limited. During drifting, the rear tire is saturated, so the rear tire force magnitude is approximately constrained by:

$$\mu F_z$$

That means the tire has a limited force “budget.”

If more of the force budget is used sideways/perpendicular to rotate the velocity vector, less is available to accelerate the car forward.

So there is a natural coupling:

more course rotation demand $\Rightarrow$ less forward acceleration, possibly deceleration

This coupling is exactly what Section 3.3 exploits.

7. What “velocity zero-dynamics” means, very carefully

Zero-dynamics is a nonlinear-control term.

Here it means:

Assume the controlled errors are zero. Then what happens to the state that was not directly controlled?

The controlled errors are:

$$e = 0$$

lateral path error zero;

$$\Delta\phi = 0$$

course error zero;

$$e_{\beta} = \beta - \beta_{ref} = 0$$

sideslip error zero;

$$e_r = r - r_{syn} = 0$$

yaw-rate synthetic tracking error zero.

If all those errors are zero, the controller is doing its job perfectly.

But $\dot{V}$ was not directly controlled.

So the remaining question is:

When $e, \Delta\phi, e_{\beta}, e_r = 0$, what does V do?

That remaining speed behavior is the **velocity zero-dynamics**.

It is called “zero” dynamics because the controlled output errors are set to zero.

8. At zero error, why $\dot{\phi}_{des} = V\kappa_{ref}$

At zero path error:

$$e = 0$$

and zero course error:

$$\Delta\phi = 0$$

the vehicle’s velocity vector is tangent to the reference path.

For a path with curvature κ_{ref} , the required course-rate is:

$$\dot{\phi}_{ref} = V\kappa_{ref}$$

This is just the basic curvature relation:

$$\text{turning rate} = \text{speed} \times \text{curvature}$$

For example, if the path is a circle of radius R , then:

$$\kappa = \frac{1}{R}$$

and:

$$\dot{\phi} = \frac{V}{R} = V\kappa$$

So at zero tracking error:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

This is the key equation in Section 3.3.

It means speed appears inside the path-tracking command even though speed is not directly controlled.

The paper makes exactly this point in Eq. 14: at zero error, the desired course-rate becomes the reference course-rate, which reduces to $V\kappa_{ref}$.  ds_142_02_021004 (2)

9. How speed becomes self-correcting

Now imagine the vehicle is drifting on a fixed-curvature path.

Suppose the car is at the correct speed V^* . Then:

$$\dot{\phi}_{des} = V^*\kappa_{ref}$$

Now suppose the car becomes too fast:

$$V > V^*$$

Then the commanded course-rate becomes larger:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

Since V is larger, $\dot{\phi}_{des}$ is larger.

So the controller says:

“Because I am moving faster along the same curved path, I need to rotate my velocity vector faster.”

To rotate the velocity vector faster, the inverse model selects steering/rear-thrust commands that generate more perpendicular acceleration.

On the correct branch of the feasible surface, this causes:

$$\dot{V} < 0$$

So the vehicle slows down.

That is negative feedback.

Now suppose the car becomes too slow:

$$V < V^*$$

Then:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

is smaller.

The controller asks for less velocity-vector rotation. On the correct branch, the resulting $\dot{V}$ becomes positive:

$$\dot{V} > 0$$

So the vehicle speeds up.

Therefore:

$$V > V^* \Rightarrow \dot{V} < 0$$

$$V < V^* \Rightarrow \dot{V} > 0$$

That is speed stability without an explicit speed controller.

This is why the paper says velocity should be stable if:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

and if $\dot{V}$ crosses zero along constant- $\dot{r}$ contours.

 ds_142_02_021004 (2)

10. Why the slope condition is $\partial \dot{V} / \partial \dot{\phi} < 0$

At zero error:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

For a fixed path curvature, larger speed means larger desired course-rate.

So the speed feedback path is:

$$V \rightarrow \dot{\phi}_{des} \rightarrow \dot{V}$$

For this feedback to be stable, increasing V should make $\dot{V}$ smaller.

Since increasing V increases $\dot{\phi}_{des}$, the condition becomes:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

This means:

If the desired velocity-vector rotation rate goes up, the resulting speed acceleration should go down.

That is exactly the right sign for speed stability.

11. Why this condition is evaluated along constant $\dot{r}$

The controller is commanding two things:

$$\dot{\phi}_{des}$$

and:

$$\dot{r}_{des}$$

So when we ask:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}}$$

we should not let $\dot{r}$ change at the same time.

We should ask:

If I keep the yaw acceleration command fixed, and only change the course-rate command, how does $\dot{V}$ change?

That is why the paper discusses $\dot{\phi}$ versus $\dot{V}$ along contours of constant $\dot{r}$, as shown in Fig. 5.

ds_142_02_021004 (2)

This is very important. The controller is not free to change $\dot{r}$ arbitrarily because $\dot{r}$ is being used to stabilize sideslip.

So speed stability is checked while preserving the yaw/sideslip objective.

12. The feasible surface: the best mental picture

This is probably the most important picture in Section 3.3.

At one instant, suppose the vehicle state is fixed:

$$V, \beta, r$$

Now imagine trying many steering angles:

$$\delta$$

and many rear thrust angles:

$$\gamma$$

For every pair (δ, γ) , the nonlinear vehicle model predicts:

$$\dot{\phi}, \dot{r}, \dot{V}$$

So every actuator pair gives one point in a 3D space:

$$(\dot{\phi}, \dot{r}, \dot{V})$$

If you sweep over all possible actuator pairs, you get a 2D surface in this 3D derivative space.

That is the "surface of possible state derivatives" in Fig. 4.

It is the set of all instantaneous motions the car can physically create from its current state.

So the nonlinear inversion is equivalent to:

Find a point on this surface whose $\dot{\phi}$ coordinate equals $\dot{\phi}_{des}$, and whose $\dot{r}$ coordinate equals $\dot{r}_{des}$. Then use the actuator pair that generated that point.

The remaining coordinate of that point is:

$$\dot{V}$$

The controller did not choose $\dot{V}$. It simply receives whatever $\dot{V}$ comes with the chosen point on the surface.

13. Why the feasible surface is 2D, not 3D

The surface is 2D because the controller has two inputs.

Input 1:

$$\delta$$

Input 2:

γ

A two-dimensional input space maps into a three-dimensional derivative space:

$$(\delta, \gamma) \mapsto (\dot{\phi}, \dot{r}, \dot{V})$$

So the possible derivative triples form a surface, not a full volume.

This means the vehicle cannot independently choose arbitrary:

$$\dot{\phi}, \dot{r}, \dot{V}$$

It can only choose combinations lying on that feasible surface.

That is why speed is not independently controllable once $\dot{\phi}$ and $\dot{r}$ are specified.

14. Why the inverse can have multiple solutions

Now imagine looking down on the feasible surface from above, onto the $(\dot{\phi}, \dot{r})$ plane.

The controller chooses a desired point:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

But the feasible surface may be folded.

So the same $(\dot{\phi}, \dot{r})$ projection may correspond to two different points on the full surface, with different $\dot{V}$ values.

That means:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

could be achieved by two different actuator pairs:

$$(\delta_1, \gamma_1)$$

and:

$$(\delta_2, \gamma_2)$$

Both might give the desired course-rate and yaw acceleration.

But one could give:

$$\dot{V} < 0$$

and the other could give:

$$\dot{V} > 0$$

This is why Section 3.3 is not merely “solve two equations.”

It must solve the equations on the correct branch.

15. The separatrix: what it really means

The separatrix is the boundary where the inverse mapping folds.

The mapping of interest is:

$$(\delta, F_{xr}) \mapsto (\dot{\phi}, \dot{r})$$

or equivalently:

$$(\delta, \gamma) \mapsto (\dot{\phi}, \dot{r})$$

The Jacobian of this mapping is:

$$J = \begin{bmatrix} \frac{\partial \dot{\phi}}{\partial \delta} & \frac{\partial \dot{\phi}}{\partial F_{xr}} \\ \frac{\partial \dot{r}}{\partial \delta} & \frac{\partial \dot{r}}{\partial F_{xr}} \end{bmatrix}$$

The determinant is:

$$\det(J) = \frac{\partial \dot{\phi}}{\partial \delta} \frac{\partial \dot{r}}{\partial F_{xr}} - \frac{\partial \dot{\phi}}{\partial F_{xr}} \frac{\partial \dot{r}}{\partial \delta}$$

The separatrix is where:

$$\det(J) = 0$$

This is Eq. 15 in the paper.

[ds_142_02_021004 \(2\)](#)

Physically, this means:

At the separatrix, steering and rear thrust no longer give two independent ways to change $\dot{\phi}$ and $\dot{r}$. The local inverse becomes singular.

Away from the separatrix, small actuator changes can independently adjust $\dot{\phi}$ and $\dot{r}$.

At the separatrix, the surface folds. The inversion becomes ambiguous or ill-conditioned.

So the separatrix is not a road boundary, not a friction circle boundary, and not directly a stability boundary.

It is an **input-output invertibility boundary**.

But it helps identify which branch of the feasible surface should be used.

16. Why the authors choose the “upper” surface

The paper says that if the separatrix lies below the plane:

$$\dot{V} = 0$$

then constraining the inversion to the “upper” surface gives stable velocity zero-dynamics and also makes the mapping one-to-one. [ds_142_02_021004 \(2\)](#)

This sentence is dense.

Here is what it means.

The feasible surface has branches. On one branch, along constant- $\dot{r}$ curves, the slope has the desired sign:

$$\frac{\partial \dot{V}}{\partial \phi} < 0$$

On the other branch, the slope may have the wrong sign or the inverse may be multivalued.

The separatrix divides these branches.

So the authors say:

We restrict the inversion to the branch above the separatrix, because on that branch speed behaves like a stable internal dynamic and the inverse map is one-to-one.

“One-to-one” means:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

corresponds to one actuator command, not multiple possible actuator commands.

That matters because if the inverse map were not one-to-one, the numerical solver could jump between branches and produce discontinuous steering/thrust commands.

17. A concrete example of the speed-stabilizing branch

Imagine the car is drifting around a constant-radius left turn.

Let:

$$\kappa_{ref} = 0.1 \text{ m}^{-1}$$

Suppose the desired drift equilibrium speed is:

$$V^* = 10 \text{ m/s}$$

Then the required course-rate is:

$$\dot{\phi}_{des} = V^* \kappa_{ref} = 1.0 \text{ rad/s}$$

At this point, suppose:

$$\dot{V} = 0$$

Now the car accidentally speeds up to:

$$V = 12 \text{ m/s}$$

Then the controller computes:

$$\dot{\phi}_{des} = 12 \times 0.1 = 1.2 \text{ rad/s}$$

So it asks for more velocity-vector rotation.

On the correct feasible-surface branch, producing this larger $\dot{\phi}$ requires more lateral/perpendicular force usage and produces:

$$\dot{V} < 0$$

The car slows down.

Now suppose the car slows to:

$$V = 8 \text{ m/s}$$

Then:

$$\dot{\phi}_{des} = 8 \times 0.1 = 0.8 \text{ rad/s}$$

The controller asks for less course rotation. On the correct branch, the selected actuator combination gives:

$$\dot{V} > 0$$

The car speeds up.

So the speed equilibrium is stable even though there is no explicit command:

$$V \rightarrow V_{ref}$$

This is the core idea.

18. Why this would not work on the wrong branch

Now imagine the inverse solver accidentally chooses the other branch.

Then increasing $\dot{\phi}_{des}$ might produce:

$$\dot{V} > 0$$

Now if the car becomes too fast:

$$V \uparrow$$

then:

$$\dot{\phi}_{des} = V \kappa_{ref} \uparrow$$

and on the wrong branch:

$$\dot{V} \uparrow$$

So the car accelerates even more.

That is positive feedback:

$$V \uparrow \Rightarrow \dot{\phi}_{des} \uparrow \Rightarrow \dot{V} > 0 \Rightarrow V \uparrow \text{ more}$$

That would be unstable.

So the branch choice is not a minor numerical detail. It is essential to making the controller work.

19. Why Section 3.3 says velocity is stabilized by coupling

The authors' big conceptual claim is:

In normal driving, steering controls path, throttle controls speed, and yaw/sideslip is naturally stable. In drifting, steering and throttle together control path and sideslip, and speed becomes naturally stable through coupling.

The paper explicitly describes this as a “shuffling of objectives”: normal driving uses steering for path and throttle for speed, while drifting uses steering and throttle together for path and sideslip, with coupled dynamics yielding stable velocity response.  ds_142_02_021004 (2)

This is the philosophical core of the paper.

They are not trying to remove the coupling.

They are using it.

That is why the paper is important: it treats the unstable drifting region not as something to avoid, but as a different control regime with a different input-output structure.

20. What Section 3.3.1 adds: what if the desired point is impossible?

So far we assumed the desired pair:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

lies on the feasible surface.

But sometimes it does not.

The controller may ask for a course-rate and yaw acceleration combination that no steering/rear-thrust command can produce.

This can happen due to:

- steering limits,
- tire friction limits,
- limited rear force direction,
- current speed too high or too low,
- current sideslip/yaw state making the desired correction too aggressive.

Section 3.3.1 says they handle this simply.

First, they saturate:

$$\dot{\phi}_{des}$$

so it falls within feasible values.

Then they compute the corresponding yaw/sideslip command.

Then they project the pair:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

into the feasible space along:

$$\dot{r} = \dot{r}_{des}$$

That means they try to preserve the yaw acceleration command as much as possible.

Why?

Because yaw/sideslip dynamics are unstable and dangerous during drifting.

If you cannot satisfy both path tracking and sideslip stabilization, you should prioritize sideslip/yaw stabilization. The paper says this projection strategy roughly prioritizes the unstable yaw/sideslip dynamics over lateral-error dynamics.  ds_142_02_021004 (2)

In plain language:

If the car cannot do everything, first keep the drift from spinning out, then fix lateral path error later.

That is a reasonable choice.

A temporary path error is recoverable.

A lost yaw/sideslip state may not be.

21. How Section 3.3 connects to Section 4

Section 3.3 gives a desired rear thrust angle or rear force combination.

But the actual car does not directly accept “rear thrust angle” as an actuator command.

The rear thrust angle is created by rear tire slip direction.

That is why Section 4 maps the desired thrust angle to a desired wheel speed.

The rear sliding tire force direction is controlled by the slip velocity vector. Changing wheel speed changes that slip vector, which changes the rear force direction.

The paper then uses a low-level wheelspeed controller so that the force/thrust command from Section 3.3 is actually realized quickly enough.  ds_142_02_021004 (2)

This matters because without wheelspeed control, the rear force response lags. The paper’s experiments show that with wheelspeed control the vehicle tracks well, while without it the yaw/sideslip and path errors become more oscillatory.  ds_142_02_021004 (2)

So Section 3.3 computes the correct desired force/thrust behavior.

Section 4 makes the real rear tires produce it.

22. One full control-step story

At one instant during the drift, the controller does this:

Step 1: Measure the vehicle state

It knows:

$$V, \beta, r, e, \Delta\phi$$

Step 2: Compute desired course-rate

Using path error dynamics, it computes:

$$\dot{\phi}_{des}$$

This says:

“How fast should the velocity vector rotate so the CG tracks the path?”

Step 3: Compute desired yaw behavior

Using sideslip error, it computes synthetic yaw rate:

$$r_{syn}$$

Then it computes desired yaw acceleration:

$$\dot{r}_{des}$$

This says:

“How fast should the vehicle body yaw acceleration be so sideslip stabilizes?”

Step 4: Nonlinear inversion

Now Section 3.3 solves:

$$\dot{\phi}_{model}(V, \beta, r, \delta, \gamma) = \dot{\phi}_{des}$$

$$\dot{r}_{model}(V, \beta, r, \delta, \gamma) = \dot{r}_{des}$$

for:

$$\delta, \gamma$$

But it solves this on the correct branch of the feasible surface.

Step 5: Accept the resulting $\dot{V}$

The model also implies:

$$\dot{V}_{model}(V, \beta, r, \delta, \gamma)$$

The controller does not command this directly.

But because the branch was selected properly, this $\dot{V}$ tends to stabilize speed.

Step 6: Map rear thrust angle to wheel speed

Section 4 maps:

$$\gamma_{des}$$

to:

$$\omega_{des}$$

Step 7: Torque control

The wheelspeed controller produces rear motor torque to track:

$$\omega_{des}$$

That is the whole controller.

23. Why the experimental results support Section 3.3

The authors later observe that the measured velocity stays close to the quasi-equilibrium values during the test, which they interpret as evidence that the closed-loop velocity dynamics are stable in the tested region. They also report good sideslip and path tracking with wheelspeed control.  ds_142_02_021004 (2)

This is important because Section 3.3 predicted:

Even without explicit speed control, speed should remain stable if the state remains in the region where the feasible-surface condition holds.

And the experiments support that.

24. The simplest possible summary of the math

The whole Section 3.3 can be reduced to this chain:

$$e, \Delta\phi \rightarrow \dot{\phi}_{des}$$

$$\begin{aligned}\beta - \beta_{ref} &\rightarrow \dot{r}_{des} \\ (\dot{\phi}_{des}, \dot{r}_{des}) &\rightarrow (\delta, \gamma) \\ (\delta, \gamma) &\rightarrow \dot{V}\end{aligned}$$

The first two arrows are Section 3.2.

The third arrow is nonlinear model inversion in Section 3.3.

The fourth arrow is the leftover internal speed dynamics.

The key condition is:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

because at zero path error:

$$\dot{\phi}_{des} = V \kappa_{ref}$$

Therefore:

$$V \uparrow \Rightarrow \dot{\phi}_{des} \uparrow \Rightarrow \dot{V} \downarrow$$

which stabilizes speed.

25. What you should remember from Section 3.3

The section is not mainly saying:

“We numerically invert a bicycle model.”

That is only the implementation.

The deeper message is:

“We choose path and sideslip as the controlled outputs, accept that speed becomes the internal dynamic, and then select the inverse-model branch where that internal speed dynamic is stable.”

That is why Section 3.3 is so important.

Without it, the controller would have desired motion derivatives but no actuator commands.

Without the zero-dynamics argument, leaving speed uncontrolled would seem unsafe.

Without the feasible-surface/separatrix argument, the nonlinear inversion could choose the wrong solution branch and make speed unstable.

So Section 3.3 is the part that makes the whole drifting controller physically coherent.

The best one-line interpretation is:

During drifting, use steering and rear thrust to control path and sideslip; let speed be the leftover dynamic, but c

Could you summarize your explanations into a picture like diagram or something I can follow by comparing your picture and your words explanations?

已中斷思考 >

編輯



Please double check your picture, make sure they contents are fully correct and also make sure the contents are consistent with your earlier explanations.

已思考 45s >

I double-checked the picture against the paper and against my previous explanations. The **main logic is correct and consistent**, but I would make a few corrections/clarifications so the diagram does not accidentally mislead you.

The paper's actual chain is:

$$(e, \Delta\phi, \beta, r) \rightarrow (\dot{\phi}_{des}, \dot{r}_{des}) \rightarrow (\delta, \gamma_{des}) \rightarrow \omega_{des} \rightarrow \tau$$

This matches the picture. The paper explicitly says the nonlinear model inversion maps the desired pair $(\dot{\phi}_{des}, \dot{r}_{des})$ into steering angle δ and rear thrust angle γ_{des} , then maps thrust angle to desired wheelspeed and finally torque.  ds_142_02_021004 (2)

The picture is correct in these important ways

Panel 1 is correct: it separates **where the vehicle body points** from **where the vehicle moves**. That is essential because in drifting β is large, so body heading and velocity direction are different.

Panel 2 is correct: path tracking gives a desired course-rate

$$\dot{\phi}_{des}$$

and sideslip stabilization gives a desired yaw acceleration

$$\dot{r}_{des}$$

Panel 3 is correct: Section 3.3 converts those desired derivatives into actuator-level commands.

Panel 5 is correct: for a fixed vehicle state (V, β, r) , all possible input pairs form a 2D feasible surface in

$$(\dot{\phi}, \dot{r}, \dot{V})$$

space.

Panel 7 is correct: if the desired pair is outside the feasible space, the authors use a simple projection strategy, roughly prioritizing yaw/sideslip stability over lateral-error correction.  ds_142_02_021004 (2)

The main correction I would make

The biggest thing I would correct is in **Panel 6**.

The picture says something like:

$$\text{upper branch} \rightarrow \dot{V} < 0 \quad \text{stable speed}$$

lower branch $\rightarrow \dot{V} > 0$ unstable speed

That is **too simplified**.

The correct statement is not:

$$\dot{V} < 0 \text{ always}$$

The correct statement is:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

along contours of constant $\dot{r}$, and $\dot{V}$ must have a zero-crossing. The paper states exactly this: velocity should be stable if $\partial \dot{V} / \partial \dot{\phi} < 0$ and $\dot{V}$ crosses zero along constant- $\dot{r}$ contours. [ds_142_02_021004 \(2\)](#)

So I would rewrite Panel 6 as:

Correct branch: $\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$ and $\dot{V} = 0$ crossing exists

not simply:

$$\dot{V} < 0$$

because at the stable speed itself:

$$\dot{V} = 0$$

If the car is too fast, then the selected branch should give:

$$\dot{V} < 0$$

If the car is too slow, it should give:

$$\dot{V} > 0$$

So the stable behavior is:

$$V > V^* \Rightarrow \dot{V} < 0$$

$$V < V^* \Rightarrow \dot{V} > 0$$

That is more accurate than saying the upper branch always gives negative $\dot{V}$.

Second clarification: γ is not a direct actuator

The picture says the nonlinear inversion outputs:

$$\delta, \gamma$$

That is correct, but with one important clarification:

$$\gamma$$

is the **desired rear thrust angle**, not a physical actuator like steering angle. The actual actuator is rear motor torque. The paper then maps:

$$\gamma_{des} \rightarrow \omega_{des} \rightarrow \tau$$

through the wheelspeed controller.

So the picture should say:

$$\gamma_{des} \text{ is an intermediate tire-force direction command}$$

not a direct command sent to the car.

Third clarification: the “force budget” explanation is intuitive, not the formal proof

Panel 4 explains speed stabilization using tire-force decomposition:

$$mV\dot{\phi} = F_{\perp}$$

$$m\dot{V} = F_{\parallel}$$

This is a very good intuition. But the paper's formal Section 3.3 argument is not based only on a simple force-budget picture. It is based on the shape of the feasible derivative surface and the condition:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

So Panel 4 is conceptually useful, but it should say:

“This is the intuition. The paper checks the actual condition using the feasible derivative surface.”

Corrected “most accurate” version of the key diagram logic

The diagram should be read like this:

$$\text{Path error } e, \Delta\phi \Rightarrow \dot{\phi}_{des}$$

$$\text{Sideslip error } \beta - \beta_{ref} \Rightarrow r_{syn} \Rightarrow \dot{r}_{des}$$

Then Section 3.3 does:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \Rightarrow (\delta, \gamma_{des})$$

But since only two derivatives are commanded, speed acceleration is left over:

$$\dot{V} \text{ is not directly commanded}$$

At zero tracking error:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

Therefore speed affects the desired course-rate. On the selected branch of the feasible surface:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

so:

$$V \uparrow \Rightarrow \dot{\phi}_{des} \uparrow \Rightarrow \dot{V} \downarrow$$

and speed stabilizes.

Final judgment

The picture is **mostly correct and consistent** with my earlier explanations. The only important correction is that **stable speed does not mean the selected branch always gives $\dot{V} < 0$** . It means the selected branch gives the correct negative slope:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

with a zero-crossing of $\dot{V}$. That is the precise Section 3.3 meaning.

So can you give me the final fully correct picture?

已思考 9s >

I created the corrected final diagram with the main fixes included, especially the correct condition:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

with a $\dot{V} = 0$ crossing, rather than saying the chosen branch always gives $\dot{V} < 0$.

Download the final corrected picture 

